CA1_spotify

February 12, 2024

Mohammad Mohannad Al Kawadri - Username: mohamad.mohannad.al.kawadri@nmbu.no

1 CA1: Dataframe Manipulation with Spotify Data

1.1 Introduction

Pandas is an extremely powerful tool to handle large amounts of tabular data. In this compulsory assignment, you will use Pandas to explore one of the TA's personal spotify data in depth.

Additional information: - Feel free to create additional code cells if you feel that one cell per subtask is not sufficient. - Remember, Pandas uses very efficient code to handle large amounts of data. For-loops are not efficient. If you ever have to use a for-loop to loop over the rows in the DataFrame, you have *probably* done something wrong. - Label all graphs and charts if applicable.

1.2 Task

I typically enjoy indie and rock music. I am a big fan of everything from old-fashioned rock and roll like Led Zeppelin and Jimi Hendrix, to newer indie artists like Joji and Lana Del Rey. This is why my spotify wrapped for 2023 came as quite a surprise:

Now, I'm no hater of pop music, but this was unexpected. For this assignment, you will investigate my listening habits, including a deep dive into my Ariana Grande listening habits, and try to find an answer to why she was my top artist; was there a fault in the spotify algorithm? Am I actually secretly an *Arianator*? (yes, I did have to look that up). Or am I just lying to myself about how often I listen to guilty pleasure music?

1.3 Part 1: Initial loading and exploration

1.0 Import necessary libraries: pandas, numpy, matplotlib.pyplot (other libraries such as seaborn or plotly are also allowed if you want prettier plots). It might also be a good idea to use **os** for task 2.0

[1]:
```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

1.1 Loading the data Load the dataset in the file `streaming_history_0.csv` into a Pandas DataFrame called `df_spotify_0`.

[2]:
```
df_spotify_0 = pd.read_csv('spotify_data/streaminghistory0.csv')
df_spotify_0.head()
```

[2]:

	endTime	artistName	trackName	msPlayed
0	2022-12-03 02:02	Cigarettes After Sex	Truly	30000.0
1	2022-12-03 02:02	Leonard Cohen	Take This Waltz - Paris Version	8210.0
2	2022-12-06 21:05	Vlad Holiday	So Damn Into You	37895.0
3	2022-12-06 21:05	Lorde	Team	8984.0
4	2022-12-06 21:05	Ariana Grande	Into You	1221.0

1.2 Help function Use the Python command `help` to help you understand how to use the `pd.DataFrame.head` and `pd.DataFrame.tail` methods.

[3]:
```
help(pd.DataFrame.head)
help(pd.DataFrame.tail)
```

```
Help on function head in module pandas.core.generic:

head(self: 'NDFrameT', n: 'int' = 5) -> 'NDFrameT'
    Return the first `n` rows.

    This function returns the first `n` rows for the object based
    on position. It is useful for quickly testing if your object
    has the right type of data in it.

    For negative values of `n`, this function returns all rows except
    the last `n` rows, equivalent to ``df[:-n]``.

    Parameters
    ----------
    n : int, default 5
        Number of rows to select.

    Returns
    -------
    same type as caller
        The first `n` rows of the caller object.

    See Also
    --------
    DataFrame.tail: Returns the last `n` rows.
```

```
    Examples
    --------
    >>> df = pd.DataFrame({'animal': ['alligator', 'bee', 'falcon', 'lion',
    …                     'monkey', 'parrot', 'shark', 'whale', 'zebra']})
    >>> df
          animal
    0  alligator
    1        bee
    2     falcon
    3       lion
    4     monkey
    5     parrot
    6      shark
    7      whale
    8      zebra

    Viewing the first 5 lines

    >>> df.head()
          animal
    0  alligator
    1        bee
    2     falcon
    3       lion
    4     monkey

    Viewing the first `n` lines (three in this case)

    >>> df.head(3)
          animal
    0  alligator
    1        bee
    2     falcon

    For negative values of `n`

    >>> df.head(-3)
          animal
    0  alligator
    1        bee
    2     falcon
    3       lion
    4     monkey
    5     parrot

Help on function tail in module pandas.core.generic:
```

```
tail(self: 'NDFrameT', n: 'int' = 5) -> 'NDFrameT'
    Return the last `n` rows.

    This function returns last `n` rows from the object based on
    position. It is useful for quickly verifying data, for example,
    after sorting or appending rows.

    For negative values of `n`, this function returns all rows except
    the first `n` rows, equivalent to ``df[n:]``.

    Parameters
    ----------
    n : int, default 5
        Number of rows to select.

    Returns
    -------
    type of caller
        The last `n` rows of the caller object.

    See Also
    --------
    DataFrame.head : The first `n` rows of the caller object.

    Examples
    --------
    >>> df = pd.DataFrame({'animal': ['alligator', 'bee', 'falcon', 'lion',
    …                     'monkey', 'parrot', 'shark', 'whale', 'zebra']})
    >>> df
          animal
    0  alligator
    1        bee
    2     falcon
    3       lion
    4     monkey
    5     parrot
    6      shark
    7      whale
    8      zebra

    Viewing the last 5 lines

    >>> df.tail()
       animal
    4  monkey
    5  parrot
    6   shark
    7   whale
```

```
8   zebra

Viewing the last `n` lines (three in this case)

>>> df.tail(3)
  animal
6  shark
7  whale
8  zebra

For negative values of `n`

>>> df.tail(-3)
   animal
3    lion
4  monkey
5  parrot
6   shark
7   whale
8   zebra
```

1.3 Getting an overview Print the first `five` and last `ten` rows of the dataframe. Have a quick look at which columns are in the dataset.

[4]:
```
# Print the first five rows of the DataFrame
print("First five rows:")
print(df_spotify_0.head())

# Print the last ten rows of the DataFrame
print("\nLast ten rows:")
print(df_spotify_0.tail(10))

# Print the columns in the DataFrame
print("\nColumns in the DataFrame:")
print(df_spotify_0.columns)
```

```
First five rows:
            endTime            artistName                        trackName  \
0  2022-12-03 02:02  Cigarettes After Sex                            Truly
1  2022-12-03 02:02         Leonard Cohen  Take This Waltz - Paris Version
2  2022-12-06 21:05          Vlad Holiday                 So Damn Into You
3  2022-12-06 21:05                 Lorde                             Team
4  2022-12-06 21:05         Ariana Grande                         Into You

   msPlayed
0   30000.0
1    8210.0
```

```
2   37895.0
3    8984.0
4    1221.0

Last ten rows:
                endTime         artistName                    trackName  \
11949  2023-01-02 20:58      Ariana Grande                    six thirty
11950  2023-01-02 20:58      Leonard Cohen          Thanks for the Dance
11951  2023-01-02 20:59           Des Rocs          Used to the Darkness
11952  2023-01-02 20:59  Caroline Polachek         Hit Me Where It Hurts
11953  2023-01-02 20:59  Caroline Polachek         Hit Me Where It Hurts
11954  2023-01-02 20:59  Kaizers Orchestra                   Resistansen
11955  2023-01-02 20:59           Mr.Kitty                    After Dark
11956  2023-01-02 20:59       daddy's girl  after dark x sweater weather
11957  2023-01-02 20:59       daddy's girl  after dark x sweater weather
11958  2023-01-02 20:59       daddy's girl  after dark x sweater weather

       msPlayed
11949    1699.0
11950   19483.0
11951     185.0
11952     603.0
11953     208.0
11954     208.0
11955  101447.0
11956     301.0
11957     208.0
11958     789.0

Columns in the DataFrame:
Index(['endTime', 'artistName', 'trackName', 'msPlayed'], dtype='object')
```

1.4 Formatting correctly When working with Pandas, it's very useful to have columns which contains dates in a specific format called *datetime*. This allows for efficient manipulation and analysis of time-series data, such as sorting, filtering by date or time, and resampling for different time periods. Figure out which column(s) would be appropriate to convert to datetime, if any, and if so, perform the conversion to the correct format.

[5]:
```
df_spotify_0['endTime'] = pd.to_datetime(df_spotify_0['endTime'])
df_spotify_0.head()
```

[5]:
```
              endTime            artistName                        trackName  \
0 2022-12-03 02:02:00  Cigarettes After Sex                            Truly
1 2022-12-03 02:02:00         Leonard Cohen  Take This Waltz - Paris Version
2 2022-12-06 21:05:00          Vlad Holiday          So Damn Into You
3 2022-12-06 21:05:00                 Lorde                             Team
4 2022-12-06 21:05:00         Ariana Grande                        Into You
```

```
   msPlayed
0   30000.0
1    8210.0
2   37895.0
3    8984.0
4    1221.0
```

1.5 Unique artists Find how many unique artists are in the dataset.

[6]:
```
unique_artists_count = df_spotify_0['artistName'].nunique()
print(f"Number of unique artists in the dataset: {unique_artists_count}")
```

```
Number of unique artists in the dataset: 495
```

1.6 Unique songs Find how many unique songs are in the dataset.

[7]:
```
unique_songs_count = df_spotify_0['trackName'].nunique()
print(f"Number of unique songs in the dataset: {unique_songs_count}")
```

```
Number of unique songs in the dataset: 1308
```

1.3.1 Part 1: Questions

Q1: Which columns are in the dataset?

endTime, artistName, trackName, msPlayed

Q2: What timeframe does the dataset span?

To answer this question i used this code:

earliest_date = df[‘endTime’].min()

latest_date = df[‘endTime’].max()

print(f”The dataset spans from {earliest_date} to {latest_date}.”)

The dataset spans from 2023-01-01 01:17:00 to 2023-12-07 21:17:00.

Q3: How many unique artists are in the dataset?

495

Q4: How many unique songs are in the dataset?

1308

1.4 Part 2: Working with all the data

2.0 Importing all the dataframes In Task 1, you only worked with about a month worth of data. Now, you will work with over a year worth.

In the *spotify_data* folder, there is more than just one listening record. Load each of the 14 listening records into a dataframe (1 dataframe per listening record), and concatenate them together into one large dataframe named `df`.

[8]:
```
dataframes = []

for i in range(14):
    file_path = f'spotify_data/streaminghistory{i}.csv'
    df_temp=pd.read_csv(file_path)
    dataframes.append(df_temp)

df= pd.concat(dataframes, ignore_index=True)
```

2.1 Sorting by time Datasets often aren't perfect. One example of an issue that could occur is that the time-based data might not be in chronological order. If this were to happen, the rows in your dataframe could be in the wrong order. To ensure this isn't an issue in your dataframe, you should sort the dataframe in chronological order, from oldest to newest.

[9]:
```
df = df.sort_values(by='endTime')
```

2.2 Setting a timeframe For this investigation, we are only interested in investigating listening patterns from **2023**. Remove any data not from **2023** from the DataFrame.

[10]:
```
df['endTime'] = pd.to_datetime(df['endTime'])
df = df[df['endTime'].dt.year == 2023]
```

2.3 Deleting rows Often in Data Science, you will encounter when a row entry has the value *NaN*, indicating missing data. These entries can skew your analysis, leading to inaccurate conclusions. For this task, identify and remove any rows in your DataFrame that contain NaN values. Later in the course, you might encounter other techniques of dealing with missing data, typically reffered to as *data imputation*. Here, though, you are just supposed to delete the entire rows with missing data.

[11]:
```
df = df.dropna()
```

2.4 Convert from milliseconds to seconds From `msPlayed`, create a new column `secPlayed` with the data converted from milliseconds to seconds. Then delete the column `msPlayed`.

[12]:
```
df['secPlayed'] = df['msPlayed'] / 1000
df = df.drop(columns='msPlayed')
df.head()
```

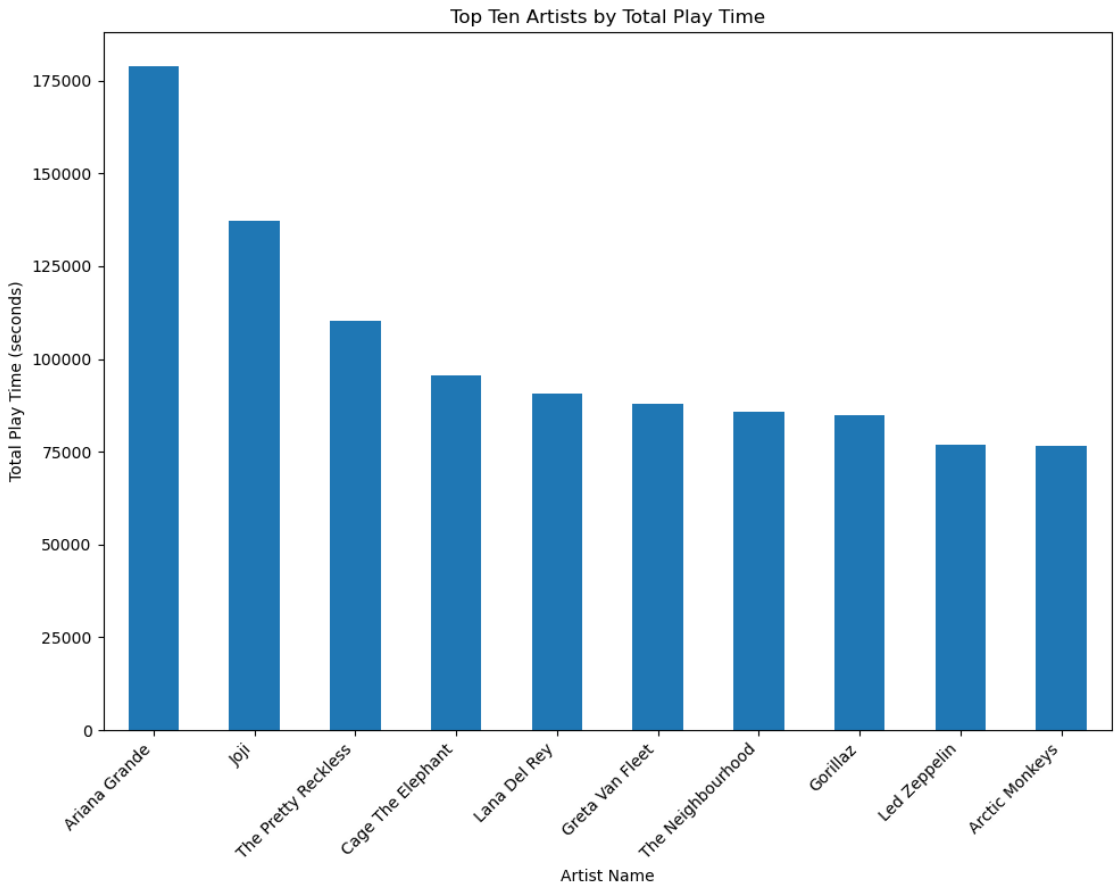
[12]:

	endTime	artistName	trackName	secPlayed
10881	2023-01-01 01:17:00	Ariana Grande	7 rings	0.139
10882	2023-01-01 01:17:00	Ariana Grande	7 rings	0.487
10883	2023-01-01 01:17:00	Ariana Grande	positions	0.417
10884	2023-01-01 01:17:00	Peach Pit	Being so Normal	2.205

```
10885 2023-01-01 01:17:00  Kelly Clarkson  Santa, Can't You Hear Me      0.278
```

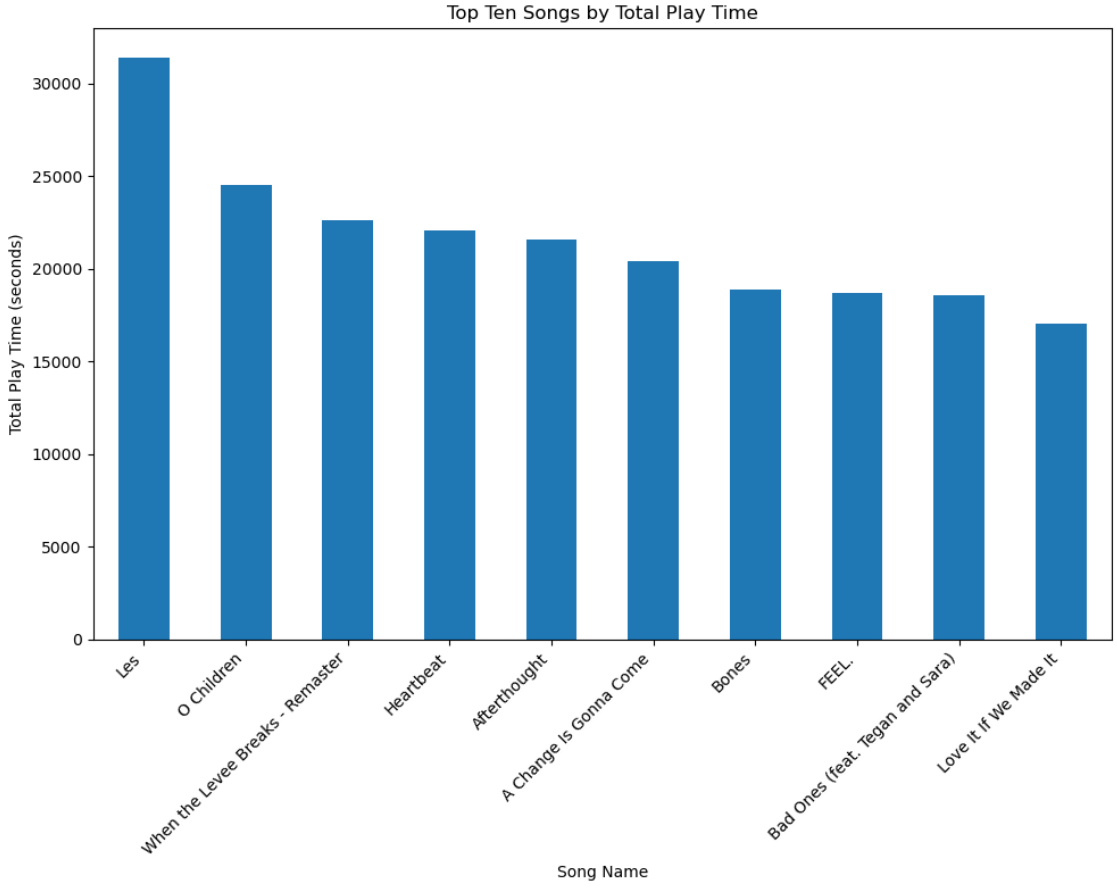
2.5 Finding top 10 favorite artists Find the top `ten` artists with the highest total play time (in seconds). Plot your findings in a bar graph.
(hint: start by creating a new DataFrame with only `artistName` and your time column. To proceed, you will also likely need the `groupby` command from Pandas.)

[13]:
```
df_artists = df[['artistName', 'secPlayed']]
artist_play_time = df_artists.groupby('artistName')['secPlayed'].sum()
top_ten_artists = artist_play_time.sort_values(ascending=False).head(10)
plt.figure(figsize=(10, 8))
top_ten_artists.plot(kind='bar')
plt.title('Top Ten Artists by Total Play Time')
plt.xlabel('Artist Name')
plt.ylabel('Total Play Time (seconds)')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



2.6 Finding top 10 favorite songs Find the top `ten` songs with the highest play time. Create a graph visualizing the results.

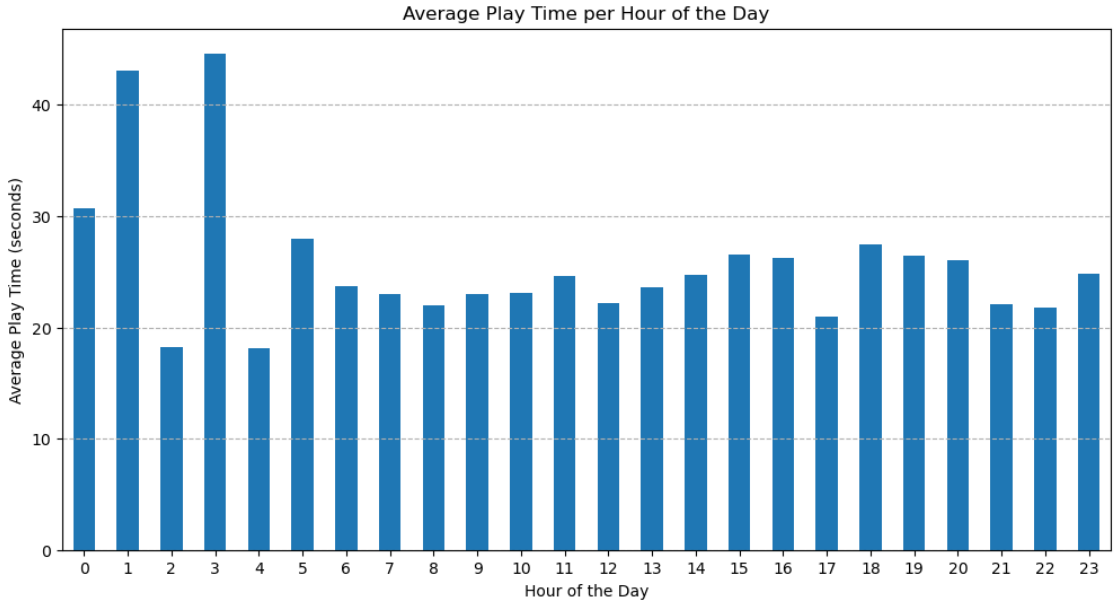
[14]:
```
df_songs = df[['trackName', 'secPlayed']]
song_play_time = df_songs.groupby('trackName')['secPlayed'].sum()
top_ten_songs = song_play_time.sort_values(ascending=False).head(10)
plt.figure(figsize=(10, 8))
top_ten_songs.plot(kind='bar')
plt.title('Top Ten Songs by Total Play Time')
plt.xlabel('Song Name')
plt.ylabel('Total Play Time (seconds)')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



1.5 Part 3: Further analysis

3.0 Average listening time by hour Generate a plot that displays the average amount of time that music is played for each hour of the day.

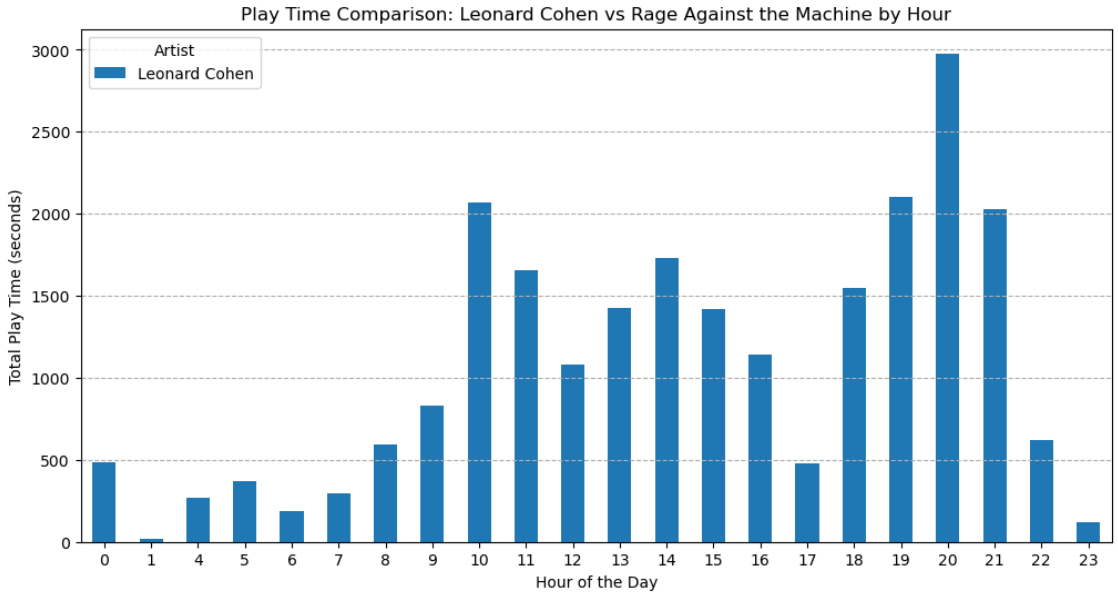
[15]:
```
df['hour'] = df['endTime'].dt.hour

average_play_time_per_hour = df.groupby('hour')['secPlayed'].mean()

plt.figure(figsize=(12, 6))
average_play_time_per_hour.plot(kind='bar')
plt.title('Average Play Time per Hour of the Day')
plt.xlabel('Hour of the Day')
plt.ylabel('Average Play Time (seconds)')
plt.xticks(range(0, 24), rotation=0)
plt.grid(axis='y', linestyle='--')
plt.show()
```



3.1 Morning music and evening music I think many people find that some types of music are more suitable for morning listening and some music is more suitable for evening listening. Create a plot that compares the play time of the artists *Leonard Cohen* and *Rage Against the Machine* on an hour-by-hour basis. See if there are any differences.

[16]:
```
df_filtered = df[df['artistName'].isin(['Leonard Cohen', 'Rage Against the␣
 ↪Machine'])].copy()
df_filtered['hour'] = df_filtered['endTime'].dt.hour
play_time_by_artist_hour = df_filtered.groupby(['artistName',␣
 ↪'hour'])['secPlayed'].sum().unstack(fill_value=0)
play_time_by_artist_hour.T.plot(kind='bar', figsize=(12, 6))
plt.title('Play Time Comparison: Leonard Cohen vs Rage Against the Machine by␣
 ↪Hour')
```

```
plt.xlabel('Hour of the Day')
plt.ylabel('Total Play Time (seconds)')
plt.xticks(rotation=0)
plt.legend(title='Artist')
plt.grid(axis='y', linestyle='--')
plt.show()
```



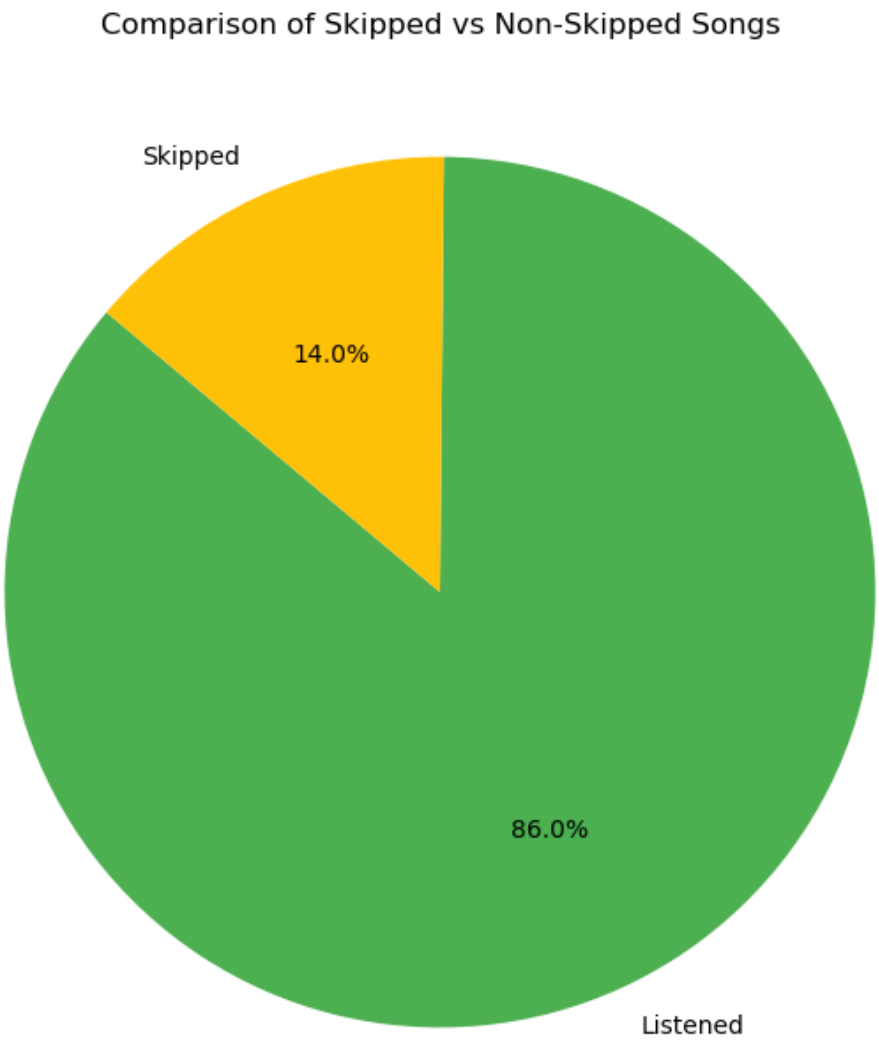
3.2 Analysing skipped songs Determining whether a song was skipped or listened to can be challenging. For this analysis, we'll simplify by defining a skipped song as any track played for less than 30 seconds. Conversely, a song played for 30 seconds or more is considered listened to.
Add a column to your DataFrame to reflect this criteria: set the value to 1 if the song was played for less than 30 seconds (indicating a skipped song), and 0 if it was played for 30 seconds or longer.

[17]:
```
df['Skipped'] = np.where(df['secPlayed'] < 30, 1, 0)
```

3.3 Plotting skipped songs Create a pie-chart that compares amount of skipped songs to amount of non-skipped songs.

[18]:
```
import matplotlib.pyplot as plt

skipped_counts = df['Skipped'].value_counts()


plt.figure(figsize=(8, 8))
plt.pie(skipped_counts, labels=['Listened', 'Skipped'], autopct='%1.1f%%',␣
 ↪startangle=140, colors=['#4CAF50', '#FFC107'])
plt.title('Comparison of Skipped vs Non-Skipped Songs')
```

```
plt.show()
```



3.4 Artists by percentage of songs skipped For each artist in the dataset, calculate which percentage of their songs was skipped. Store this information in a new DataFrame called `df_skipped`. Store the percentage of skipped songs in a new column named `SkipRate`

Example: If an artist has **100** songs in your dataset and **25** of these were skipped, the percentage of skipped songs for this artist would be $\frac{25}{100} = 25\%$

[19]:
```
artist_stats = df.groupby('artistName').agg(
    TotalSongs=('Skipped', 'size'),
```

```
    SkippedSongs=('Skipped', 'sum'))


artist_stats['SkipRate'] = (artist_stats['SkippedSongs'] /␣
 ↪artist_stats['TotalSongs']) * 100

artist_stats.reset_index(inplace=True)

df_skipped = artist_stats[['artistName', 'SkipRate']]
```

3.5 Comparing artists by skip-rate Find the `three` top artists with the lowest skip-rate and the `three` with the highest. Print their names, along with their skip-rate.

[20]:
```
top_lowest_skip_rate = df_skipped.sort_values(by='SkipRate').head(3)

top_highest_skip_rate = df_skipped.sort_values(by='SkipRate', ascending=False).
 ↪head(3)

print("Top three artists with the lowest skip-rate:")
print(top_lowest_skip_rate)

print("\nTop three artists with the highest skip-rate:")
print(top_highest_skip_rate)
```

```
Top three artists with the lowest skip-rate:
        artistName   SkipRate
305  Gloria Gaynor   0.000000
645       Roc Boyz  11.111111
437          LACES  14.285714

Top three artists with the highest skip-rate:
            artistName  SkipRate
328     Hannah Montana     100.0
28   Alexander Stewart     100.0
560        No Vacation     100.0
```

1.6 Part 4: God Is a Data Scientist - The Ariana Deep-Dive

4.0 Ariana-DataFrame: Create a new DataFrame called *df_ariana*, containing only rows with music by Ariana Grande.

[21]:
```
df_ariana = df[df['artistName'] == 'Ariana Grande']
df_ariana
```

[21]:
```
                   endTime     artistName                          trackName  \
10881  2023-01-01 01:17:00  Ariana Grande                            7 rings
10882  2023-01-01 01:17:00  Ariana Grande                            7 rings
10883  2023-01-01 01:17:00  Ariana Grande                          positions
```

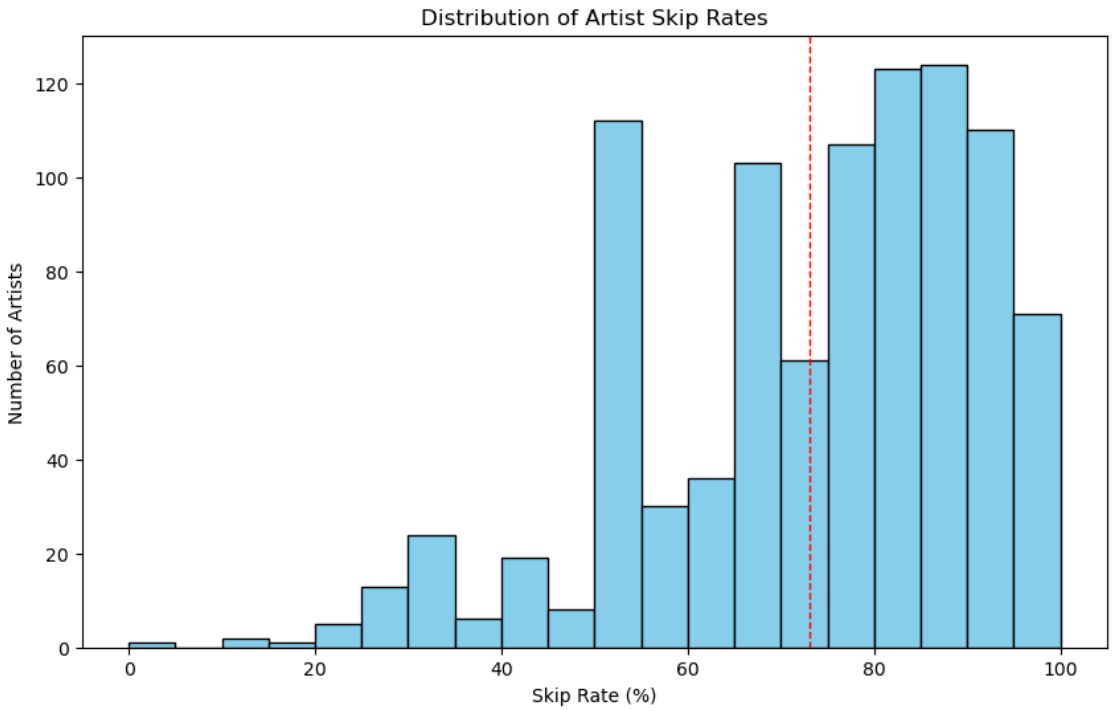
```
10887  2023-01-01 01:17:00  Ariana Grande                        Santa Baby
10888  2023-01-01 01:17:00  Ariana Grande     Right There (feat. Big Sean)
…                    …            …                                  …
167415 2023-12-07 17:46:00  Ariana Grande           Almost Is Never Enough
167422 2023-12-07 20:51:00  Ariana Grande                           needy
167428 2023-12-07 21:13:00  Ariana Grande                     pete davidson
167435 2023-12-07 21:13:00  Ariana Grande  off the table (with The Weeknd)
167436 2023-12-07 21:14:00  Ariana Grande                         my hair

        secPlayed  hour  Skipped
10881       0.139     1        1
10882       0.487     1        1
10883       0.417     1        1
10887      12.293     1        1
10888      22.929     1        1
…             …   …       …
167415     28.483    17        1
167422     26.220    20        1
167428      0.603    21        1
167435     13.448    21        1
167436     23.757    21        1

[19337 rows x 6 columns]
```

4.1 Average skip rate Create a histogram of the distribution of the skip-rate values of the different artists in your DataFrame `df_skipped`, with skip rates on one axis and number of artists on the other.

Then, retrieve the skip rate for Ariana Grande from your DataFrame `df_skipped`. Run the code in the cell below. Where on this distribution does Ariana Grande fall? Do I skip her songs more than average, or less?

[22]:
```
plt.figure(figsize=(10, 6))
plt.hist(df_skipped['SkipRate'], bins=20, color='skyblue', edgecolor='black')
plt.title('Distribution of Artist Skip Rates')
plt.xlabel('Skip Rate (%)')
plt.ylabel('Number of Artists')
plt.axvline(df_skipped['SkipRate'].mean(), color='r', linestyle='dashed',␣
 ↪linewidth=1)
plt.show()

ariana_skip_rate = df_skipped[df_skipped['artistName'] == 'Ariana␣
 ↪Grande']['SkipRate'].iloc[0]
print(f"Ariana Grande's skip rate: {ariana_skip_rate}%")

average_skip_rate = df_skipped['SkipRate'].mean()
print(f"Average skip rate: {average_skip_rate}%")
```

```
if ariana_skip_rate > average_skip_rate:
    print("Ariana Grande's songs are skipped more than average.")
elif ariana_skip_rate < average_skip_rate:
    print("Ariana Grande's songs are skipped less than average.")
else:
    print("Ariana Grande's songs are skipped at an average rate.")
```




```
Ariana Grande's skip rate: 99.52939959662822%
Average skip rate: 73.04822293282277%
Ariana Grande's songs are skipped more than average.
```

1.6.1 Part 4: Questions

Q1: Did I skip a lot of Ariana Grande's songs, or did I not, compared to the rest of the dataset?

Yes, you did skip Ariana Grande's songs with almost 99.5% skip rate.

Q2: What might be some possible reasons for Ariana Grande to be my nr.1 artist?

The reason for that is that you played many songs for Ariana, 167436 times you played Ariana songs.

[23]:
```
df[df['artistName'] == 'Ariana Grande']
```

[23]:

	endTime	artistName	trackName	secPlayed	hour	Skipped
10881	2023-01-01 01:17:00	Ariana Grande	7 rings	0.139	1	1
10882	2023-01-01 01:17:00	Ariana Grande	7 rings	0.487	1	1
10883	2023-01-01 01:17:00	Ariana Grande	positions	0.417	1	1
10887	2023-01-01 01:17:00	Ariana Grande	Santa Baby	12.293	1	1
10888	2023-01-01 01:17:00	Ariana Grande	Right There (feat. Big Sean)	22.929	1	1
…	…	…	…	…	…	…
167415	2023-12-07 17:46:00	Ariana Grande	Almost Is Never Enough	28.483	17	1
167422	2023-12-07 20:51:00	Ariana Grande	needy	26.220	20	1
167428	2023-12-07 21:13:00	Ariana Grande	pete davidson	0.603	21	1
167435	2023-12-07 21:13:00	Ariana Grande	off the table (with The Weeknd)	13.448	21	1
167436	2023-12-07 21:14:00	Ariana Grande	my hair	23.757	21	1

[19337 rows x 6 columns]

[ ]: